

Chapter 4 – Struktur Kontrol Alur

Dalam pemrograman, kita sering membutuhkan kemampuan untuk **membuat keputusan** dan **mengulang suatu proses**.

Struktur kontrol alur memungkinkan program berjalan secara **dinamis** berdasarkan kondisi tertentu.

Pada chapter ini, Anda akan mempelajari:

- Percabangan menggunakan `if`, `elif`, dan `else`
- Perulangan menggunakan `for`
- Perulangan menggunakan `while`

Tujuan Pembelajaran

Setelah mempelajari chapter ini, Anda diharapkan mampu:

- Memahami konsep kontrol alur dalam program
- Menggunakan percabangan `if-elseif-else` untuk mengambil keputusan
- Menggunakan perulangan `for`
- Menggunakan perulangan `while`
- Mengombinasikan perulangan bersarang (nested loop)
- Menggunakan `break` dan `continue` dengan tepat
- Menerapkan kontrol alur dalam program sederhana

4.1 Percabangan `if`, `elif`, dan `else`

Percabangan digunakan untuk menjalankan kode **hanya ketika** suatu kondisi terpenuhi. Tanpa percabangan, program akan selalu melakukan hal yang sama, apa pun input-nya.

4.1.1 Struktur Dasar `if`:

```
if kondisi:  
    perintah
```

kondisi adalah ekspresi yang bernilai `True` atau `False` perintah hanya dijalankan jika kondisi bernilai `True`

Contoh:

```
nilai = 80  
if nilai >= 75:  
    print("Lulus")
```

Jika `nilai >= 75` bernilai `True`, maka teks "Lulus" akan ditampilkan.

4.1.2 Menambah Pilihan dengan elif dan else

Untuk kondisi lebih dari satu, gunakan elif dan else:

```
nilai = 65
if nilai >= 80:
    print("Sangat Baik")
elif nilai >= 70:
    print("Baik")
else:
    print("Perlu Perbaikan")
```

Penjelasan: - if → dicek dulu: apakah nilai ≥ 80 ? - elif → jika kondisi if salah, cek: apakah nilai ≥ 70 ? - else → dijalankan jika **semua kondisi sebelumnya salah**

Tips

Urutan kondisi sangat penting. Python akan mengecek dari atas ke bawah, dan berhenti pada kondisi pertama yang bernilai True.

Diagram Alur if-elif-else

Ilustrasi alur keputusan:



4.1.4 Pentingnya Indentasi di Python

Python menggunakan **indentasi** (spasi di awal baris) untuk menentukan blok kode. Kode di dalam if, for, dan while harus **menjorok ke dalam**.

Contoh yang benar:

```
if nilai >= 75:
    print("Lulus")
    print("Selamat!")
```

Contoh yang salah (tanpa indentasi):

```
nilai = 80
if nilai >= 75:
    print("Lulus") # Error
```

Catatan Penting

Gunakan 4 spasi untuk satu level indentasi (standar Python). Indentasi yang salah akan menyebabkan error seperti: `IndentationError: expected an indented block`

4.2 Perulangan for

Perulangan for digunakan untuk mengulang suatu proses dalam jumlah tertentu atau untuk setiap elemen dalam sebuah urutan (seperti list, string, dan sebagainya).

4.2.1 Struktur Dasar for

```
for i in range(5):
    print(i)
```

Hasil:

```
0
1
2
3
4
```

Penjelasan: `range(5)` menghasilkan deret: 0, 1, 2, 3, 4. `i` akan berisi setiap nilai tersebut **secara bergantian**. `print(i)` dipanggil 5 kali, satu kali untuk setiap nilai `i`.

4.2.2 Contoh Perulangan dengan Awal dan Akhir

```
for i in range(1, 6):
    print("Angka:", i)
```

Hasil:

```
Angka: 1
Angka: 2
Angka: 3
Angka: 4
Angka: 5
```

Perulangan for sangat cocok digunakan ketika **jumlah pengulangan sudah diketahui**.

Tips

Gunakan for jika Anda sudah tahu **berapa kali** perulangan dilakukan, misalnya: 10 kali, 100 kali, atau sesuai panjang data.

4.3 Nested for (Perulangan Bersarang)

Nested for adalah perulangan for di dalam perulangan for yang lain.

4.3.1 Struktur Dasar

```
for i in range(...):  
    for j in range(...):  
        perintah
```

Artinya: Perulangan **luar (i)** berjalan lebih dulu Untuk **setiap nilai i**, perulangan **dalam (j)** akan berjalan **penuh**

4.3.2 Contoh sederhana:

```
for i in range(3):  
    for j in range(2):  
        print("i =", i, ", j =", j)
```

Hasil:

```
i = 0 , j = 0  
i = 0 , j = 1  
i = 1 , j = 0  
i = 1 , j = 1  
i = 2 , j = 0  
i = 2 , j = 1
```

Penjelasan alurnya:

- i = 0
 - j = 0
 - j = 1
- i = 1
 - j = 0
 - j = 1
- i = 2
 - j = 0
 - j = 1

Perulangan **dalam** selalu “habis dulu” sebelum perulangan luar lanjut ke nilai berikutnya.

4.3.3 Contoh: Membuat Pola Bintang

```
for baris in range(3):  
    for kolom in range(5):  
        print("*", end=" ")  
    print()
```

Hasil:

```
* * * * *  
* * * * *  
* * * * *
```

Penjelasan:

baris mengatur **jumlah baris** kolom mengatur **jumlah bintang di setiap baris** print("*", end=" ") menampilkan bintang **tanpa** pindah baris print() kosong digunakan untuk **pindah baris** setelah satu baris selesai

Catatan

Nested for sering digunakan untuk: Membuat tabel - Membuat pola (bintang, angka, dll.) - Mengolah data dua dimensi (baris dan kolom) - Simulasi papan permainan atau grid

Tips

Cara mudah memahami nested for adalah dengan membayangkan: "Untuk setiap baris, lakukan perulangan kolom."

Bacalah kode ini seperti kalimat:

```
for baris in range(3):  
    for kolom in range(5):  
        ...
```

Artinya: "Untuk setiap baris, ulangi proses kolom sebanyak 5 kali."

4.4 Variasi range() pada Perulangan for

Fungsi range() memiliki tiga bentuk utama:

```
range(akhir)  
range(awal, akhir)  
range(awal, akhir, langkah)
```

Bentuk ketiga memungkinkan kita mengatur **langkah (step)** perulangan.

4.4.1 Contoh 1 – Melompat dua angka:

```
for i in range(2, 10, 2):  
    print("index:", i)
```

Hasil:

```
index: 2  
index: 4  
index: 6  
index: 8
```

Penjelasan:

Mulai dari 2 Berhenti **sebelum** 10 Melompat setiap 2 langkah (2 → 4 → 6 → 8)

Contoh ini cocok untuk: Menampilkan bilangan genap Melakukan proses setiap dua langkah

```
for i in range(2, 10, 2):  
    print("index:", i)
```

4.4.2 Contoh 2 – Menghitung Mundur:

```
for i in range(5, -5, -1):  
    print("index:", i)
```

Hasil:

```
index: 5  
index: 4  
index: 3  
index: 2  
index: 1  
index: 0  
index: -1  
index: -2  
index: -3  
index: -4
```

Penjelasan:

- Mulai dari 5
- Berhenti sebelum -5

Langkah -1 artinya mundur satu per satu (5 → 4 → 3 → ... → -4)

Contoh ini cocok untuk: - Hitung mundur (countdown) - Memproses data dari besar ke kecil

Catatan

Nilai **akhir** pada range() **tidak pernah** ikut ditampilkan. Itulah sebabnya range(2, 10, 2) berhenti di 8, bukan 10.

Tips

Ingat pola berpikir range(awal, akhir, langkah) sebagai: "Mulai dari **awal***, bergerak dengan **langkah**, dan berhenti **sebelum akhir**."

4.4 Perulangan while

Perulangan while digunakan selama suatu **kondisi masih bernilai True**. Berbeda dengan for yang jumlah pengulangannya jelas, while lebih cocok saat jumlah pengulangan belum pasti.

4.5.1 Struktur Dasar:

```
while kondisi:  
    perintah
```

Contoh:

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

Hasil:

```
1  
2  
3  
4  
5
```

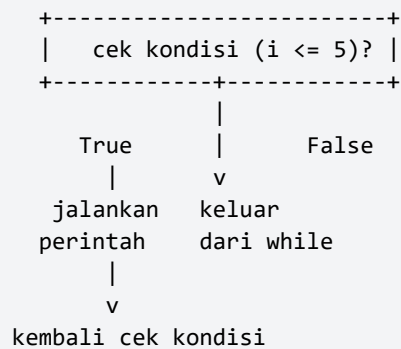
Penjelasan:

Awalnya, $i = 1$ Selama $i \leq 5$ bernilai True, blok di dalam while akan dijalankan $i += 1$ akan menambah nilai i setiap putaran Saat i menjadi 6, kondisi $i \leq 5$ menjadi False dan perulangan berhenti Perulangan while cocok digunakan ketika:

Jumlah pengulangan **bergantung pada kondisi**, bukan angka pasti Misalnya: terus meminta input sampai user memasukkan nilai yang benar

4.5.2 Flowchart Sederhana while

Ilustrasi alur while:



Catatan

Perulangan while cocok digunakan saat: Menunggu kondisi tertentu terpenuhi Terus meminta input sampai user mengisi dengan benar Proses yang durasinya belum diketahui di awal

4.5.3 Waspada Infinite Loop

Hati-hati dengan **infinite loop** (perulangan tak berujung).

Contoh berbahaya:

```

i = 1
while i <= 5:
    print(i)
  
```

Karena **i tidak pernah berubah**, kondisi selalu True dan program tidak akan berhenti. Selalu pastikan variabel yang muncul dalam kondisi ($i \leq 5$) **diubah** di dalam loop.

Peringatan

Selalu pastikan ada bagian di dalam while yang mengubah kondisi, misalnya $i += 1$, sehingga suatu saat kondisi menjadi False.

4.6 Nested while

Nested while adalah perulangan while di dalam perulangan while lain.

```

i = 1
while i <= 3:
    j = 1
    while j <= 2:
        print("i =", i, ", j =", j)
        j += 1
    i += 1
  
```


Hasil:

```
i = 1 , j = 1
i = 1 , j = 2
i = 2 , j = 1
i = 2 , j = 2
i = 3 , j = 1
i = 3 , j = 2
```

Penjelasan: i berjalan dari 1 sampai 3 Untuk setiap nilai i, j selalu diulang dari 1 sampai 2

Catatan

Pada nested while, pastikan **setiap variabel** loop berubah (i += 1, j += 1). Jika salah satu tidak berubah, program bisa terjebak **infinite loop**.

4.7 Menghentikan atau Melewati Perulangan: break dan continue

Python menyediakan perintah khusus:

- break → **menghentikan** seluruh perulangan
- continue → **melewati satu iterasi** (putaran) saat ini dan lanjut ke putaran berikutnya

4.7.1 break pada for

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Hasil:

```
1
2
3
4
```

Penjelasan:

Saat i == 5, perintah break dijalankan Perulangan langsung **berhenti**, sehingga angka 5 dan seterusnya tidak dicetak

4.7.2 continue pada for:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Hasil:

```
1
2
4
5
```

Penjelasan:

Saat `i == 3`, perintah `continue` dijalankan Python melewati sisa kode di dalam loop untuk putaran itu, lalu lanjut ke `i` berikutnya Hasilnya, angka 3 tidak dicetak ::: {tip title="Tips Penggunaan break"} Gunakan `break` jika ingin menghentikan proses lebih awal, misalnya ketika **data yang dicari sudah ditemukan** dan tidak perlu melanjutkan perulangan. :::

4.7.3 Menggabungkan while dan for

```
i = 1
while i <= 3:
    for j in range(1, 4):
        print("i =", i, ", j =", j)
    i += 1
```

Struktur ini sering dipakai saat: - Perulangan luar (`while`) bergantung pada **kondisi** - Perulangan dalam (`for`) jumlahnya **sudah pasti** * ## 4.8 `break` dan `continue` pada `while`

4.8.1 `break` pada `while`:

```
i = 1
while i <= 10:
    if i == 5:
        break
    print(i)
    i += 1
```

Hasil:

```
1
2
3
4
```

Penjelasan:

Saat `i` bernilai 5, `break` menghentikan perulangan `while` Perulangan tidak pernah mencapai 5, 6, ... , 10

4.8.2 continue pada while:

```
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue
    print(i)
```

Hasil: 1 2 4 5

Penjelasan:

i dimulai dari 0 Di awal loop, i selalu ditambah 1 Saat $i == 3$, continue membuat Python **melewati** `print(i)` Angka 3 tidak pernah dicetak ::: {tip title="Ringkasan"}

`break` → menghentikan **seluruh perulangan** continue → melewati **satu putaran saja**, lalu lanjut ke putaran berikutnya Prinsipnya sama pada for dan while. Yang berbeda hanya **bentuk perulangannya**. :::

4.9 Latihan

Percabangan Nilai

Buat program yang meminta input nilai dari pengguna. Jika nilai ≥ 75 , tampilkan "Lulus", selain itu "Tidak Lulus".

Perulangan for Buat perulangan for untuk menampilkan angka 1 sampai 10. Perulangan while

Perulangan while Buat perulangan while yang berhenti ketika angka mencapai 5.

Contoh solusi:

```
nilai = int(input("Masukkan nilai: "))

if nilai >= 75:
    print("Lulus")
else:
    print("Tidak Lulus")

for i in range(1, 11):
    print(i)

i = 1
while i <= 5:
    print(i)
    i += 1
```

Latihan Tambahan – continue

Buat program yang menampilkan angka 1 sampai 10, tetapi **melewati angka 5** (tidak ditampilkan).

Petunjuk: - Gunakan perulangan for - Gunakan continue untuk melewati angka tertentu

Contoh keluaran yang diharapkan:

```
1
2
3
4
6
7
8
9
10
```

Contoh solusi:

```
for i in range(1, 11):
    if i == 5:
        continue
    print(i)
```

Catatan

Perintah `continue` tidak **menghentikan perulangan**, tetapi hanya melewati **satu putaran (iterasi)** saja.

Pada chapter berikutnya, Anda akan mempelajari **Fungsi**, yaitu cara membuat blok kode yang dapat digunakan kembali dalam program Python.